

- ◆ **Établissement : Faculté des Sciences Semlalia de Marrakech (FSSM)**
- ◆ **Département : Informatique**
- ◆ **Année universitaire : 2024-2025**

Mini-Projet de Programmation Orientée Objet en C++

Application de jeu de course à vélo en C++ avec SDL2

- **Réalisé par :**
 - Saloumy Meriem
 - Doubabi Ali
- **Encadré par :** Prof: R HANNANE
- **Date de remise :** 10 mai 2025

1. Introduction

Le mini-projet présenté dans ce rapport s'inscrit dans le cadre du module de Programmation Orientée Objet en langage C++. Il a pour objectif de concevoir et développer une application ludique : un jeu de course à vélo.

Ce projet a permis de mettre en pratique les connaissances acquises durant le semestre, notamment la conception orientée objet, la structuration d'un projet logiciel, ainsi que la gestion d'un projet en binôme. De plus, il a introduit l'utilisation d'une bibliothèque graphique externe, SDL2, qui est largement utilisée dans le domaine des jeux vidéo.

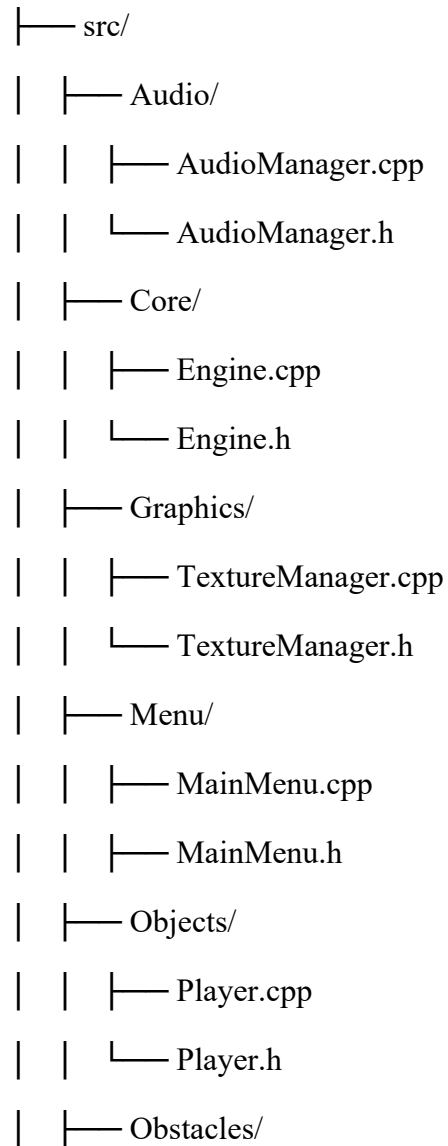
L'application développée est un jeu 2D interactif dans lequel l'utilisateur contrôle un cycliste qui doit terminer un parcours dans un temps limité sans heurter les obstacles (murs). Le jeu propose une interface graphique simple, un menu principal avec différentes options, et des interactions clavier pour contrôler le vélo.

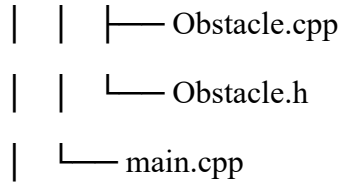
Ce rapport détaille les étapes de conception de l'application, la structure des classes, les choix techniques et les principales fonctionnalités. Il met également en évidence l'architecture logicielle orientée objet utilisée pour modéliser les différents éléments du jeu.

Enfin, ce projet nous a offert une expérience concrète de développement logiciel en équipe, tout en nous familiarisant avec les notions de graphisme 2D et la gestion d'un code modulaire en C++.

Arborescence du projet :

GameEngine_2D/





Specifications des besoins :

Le jeu développé est une application ludique en 2D, basée sur un moteur simple construit avec la bibliothèque SDL2. Il repose sur les fonctionnalités de base suivantes :

- **Menu principal** avec les options suivantes :
 - Jouer
 - À propos
 - Quitter
- **Menu de pause** accessible en cours de jeu avec les options :
 - Reprendre la partie
 - Quitter le jeu
- **Boucle de jeu** qui gère les entrées clavier, les mises à jour d'état, les collisions et le rendu.
- **Affichage du joueur** pouvant être déplacé à l'aide du clavier.
- **Apparition aleatoire d'obstacles** sur la scène de jeu que le joueur doit éviter.
- **Chronomètre** pour suivre la durée de jeu : Le jeu utilise un **compte à rebours visuel** qui commence par une image "start", puis affiche successivement les chiffres de 60 à 0, en décrémentant chaque seconde. Une fois le temps écoulé,

une image "gameover" s'affiche pour signaler la fin de la course.

- **Gestion du son** (musique de fond et effets sonores) via un gestionnaire audio.
- **Système de progression et condition de victoire :**
Affichage de la distance parcourue en temps réel (ex. 1200m / 3500m) avec victoire si le joueur atteint 3500m avant la fin du temps.

Structure des Classes :

Classe : Engine

Rôle : Gère le cycle de vie du jeu (initialisation, boucle principale, rendu, événements), les états du jeu, la progression du joueur, l'audio, les obstacles, et l'interface utilisateur.

Attributs :

1. Variables de gestion du son :

- `m_currentMasterVolume` : Volume du jeu (maximum de 128).
- `m_isMuted` : Indique si le jeu est en mode muet.
- `m_volumeBeforeMute` : Volume avant que le jeu ne soit mis en mode muet.

2. Variables de gestion de l'état du jeu :

- `m_IsRunning` : Indique si le jeu est en cours d'exécution.
- `m_Window`, `m_Renderer` : Pointeurs vers la fenêtre et le rendu SDL.

- `m_Player` : Pointeur vers l'objet `Player` qui représente le joueur.
- `m_gameState` : L'état actuel du jeu (par exemple, écran principal, jeu en cours, pause, etc.).

3. Variables liées au gameplay :

- `m_remainingSeconds` : Nombre de secondes restantes avant la fin du jeu.
- `m_totalDistanceTraveled` : Distance totale parcourue par le joueur.
- `m_obstacleSpawnInterval`,
`m_timeSinceLastSpawn` : Intervalle de temps pour la génération des obstacles et temps écoulé depuis le dernier obstacle.
- `m_maxSpeedIncreaseAmount` : Augmentation de la vitesse maximale du joueur.

4. Variables d'interface utilisateur et visuels :

- `m_uiFont` : Police utilisée pour le texte de l'interface.
- `m_distanceTexture` : Texture utilisée pour afficher la distance parcourue.
- `m_returnPromptTexture` : Texture pour l'affichage d'un message de retour.
- `m_timerTextures` : Liste des textures pour l'affichage du chronomètre.
- `m_timerRect` : Position et taille du chronomètre à l'écran.

5. Variables liées à la gestion des obstacles :

- `m_obstacles` : Liste d'obstacles générés dans le jeu.
- `m_obstacleTextureWidth`,
`m_obstacleTextureHeight` : Dimensions des textures des obstacles.
- `m_obstacleTextureIds` : Identifiants des textures des obstacles.

Méthodes :

1. Gestion du jeu :

- `Init()` : Initialise le moteur du jeu, la fenêtre, et le rendu.
- `Clean()` : Nettoie les ressources avant de quitter.
- `Quit()` : Ferme le jeu et libère les ressources.
- `Update()` : Met à jour l'état du jeu à chaque frame.
- `Render()` : Effectue le rendu des éléments du jeu à l'écran.
- `Events()` : Gère les événements (comme les entrées utilisateur).
- `HandleEvents(SDL_Event& e)` : Gère les événements spécifiques du jeu.

2. Contrôle du son :

- `IncreaseVolume()` : Augmente le volume du jeu.

- `DecreaseVolume()` : Diminue le volume du jeu.
- `ToggleMute()` : Bascule entre le mode muet et le mode normal.
- `ApplyMasterVolume()` : Applique le volume du jeu.
- `IsMuted()` : Retourne vrai si le jeu est muet.

3. Gestion de l'état du jeu :

- `GetGameState()` : Retourne l'état actuel du jeu.
- `SetGameState(GameState newState)` : Change l'état du jeu.
- `TogglePause()` : Bascule entre la pause et le jeu actif.
- `RenderPauseMenu()` : Affiche le menu de pause.
- `ResetGameData()` : Réinitialise les données du jeu.

4. Gestion des obstacles :

- `SpawnObstacle()` : Génère un obstacle sur la scène.

5. Gestion des menus :

- `RenderReturnPrompt()` : Affiche un message de retour à l'écran.
- `ToggleReturnPrompt()` : Affiche ou cache le message de retour.

- Elle permet de charger, dessiner, interroger les dimensions, libérer et nettoyer les textures.
- Les textures sont associées à des identifiants afin de les gérer facilement.

Environnement de Travail :

- Bibliothèques : SDL2, SDL_image, SDL_ttf,SDL_mixer.
- IDE : CodeBlocks.
- Système d'exploitation : Windows .
- Outils : GitHub pour la gestion de version.

Interfaces de Jeu :

Figure du menu principale :



Effet visuel lors des cliques sur les boutons :



En cours du jeu :



Écran de Pause

